

Assignment 03: Neural Networks and Representation Learning

“The important thing is not only to make a model learn, but to understand what the model is learning.”

Intelligent System Development

Neural Networks and Representation Learning

Dinh Que Tran, Ph.D., Assoc. Prof.

Department of Information Technology

Posts and Telecommunications Institute of Technology

quetd@ptit.edu.vn

From Assignment 02 to Assignment 03

Assignment 02 focused on traditional machine learning.

Data → Prepared Features → ML Algorithm → Prediction

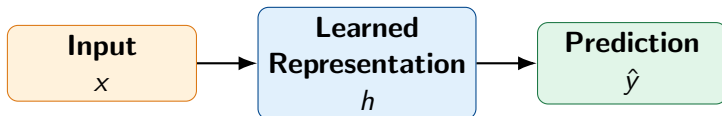
Assignment 03 asks a deeper question:

Can the model learn useful representations from the data?

Data → Learned Representation → Prediction

The Central Question

What does a neural network actually learn?



The representation h is not manually specified. Instead, its parameters are learned from data.

$$x \rightarrow h_{\theta}(x) \rightarrow \hat{y}$$

where θ denotes the learnable parameters.

What Is a Neural Network?

A neural network is a parameterized function composed of successive transformations.

A simple network can be written as

$$f_{\theta}(x) = W_2 \phi(W_1 x + b_1) + b_2.$$

The network contains:

- weights W ;
- biases b ;
- activation functions ϕ ;
- learnable parameters θ .

Key Idea

A neural network is not magic.

It is a parameterized mathematical function whose parameters are learned from data.

A Neuron

A single neuron computes:

$$z = w^T x + b.$$

An activation function then produces

$$h = \phi(z).$$



The important distinction is:

Linear transformation + Nonlinearity

Linear Transformation

For an input vector

$$x \in \mathbb{R}^d,$$

a linear layer computes

$$z = Wx + b.$$

If

$$W \in \mathbb{R}^{k \times d},$$

then

$$z \in \mathbb{R}^k.$$

Therefore:

$$\boxed{\mathbb{R}^d \rightarrow \mathbb{R}^k}$$

Why Do We Need Activation Functions?

Suppose every layer were purely linear:

$$h_1 = W_1 x + b_1$$

and

$$h_2 = W_2 h_1 + b_2.$$

Then the composition is still a linear-affine transformation:

$$h_2 = W_2 W_1 x + W_2 b_1 + b_2.$$

Therefore, stacking linear layers alone does not provide the expressive power expected from a deep network.

Depth + Nonlinearity $\rightarrow$ Expressive Model

The ReLU Activation Function

A common activation function is the Rectified Linear Unit (ReLU):

$$\text{ReLU}(z) = \max(0, z).$$

Equivalently,

$$\text{ReLU}(z) = \begin{cases} 0, & z < 0, \\ z, & z \geq 0. \end{cases}$$

Interpretation

ReLU introduces a nonlinear transformation: negative values are mapped to zero, while positive values are preserved.

In PyTorch:

```
nn.ReLU()
```

From One Neuron to a Layer

Instead of one neuron, a layer contains many neurons.

$$z = Wx + b$$

where

$$W \in \mathbb{R}^{k \times d}, \quad b \in \mathbb{R}^k.$$

Then

$$h = \phi(z).$$

Conceptually:

$$x \in \mathbb{R}^d \rightarrow h \in \mathbb{R}^k$$

Each output dimension corresponds to a learned transformation of the input.

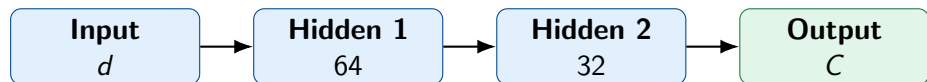
A Multi-Layer Perceptron

A simple MLP can be represented as:

$$x \rightarrow W_1x + b_1 \rightarrow \text{ReLU} \rightarrow W_2h + b_2 \rightarrow \hat{y}$$

For example:

$$d \rightarrow 64 \rightarrow 32 \rightarrow C.$$



The Hidden Representation

Consider

$$h_1 = \text{ReLU}(W_1x + b_1).$$

The vector h_1 is a learned representation.

$$x \in \mathbb{R}^d \quad \longrightarrow \quad h_1 \in \mathbb{R}^{64}.$$

Unlike Assignment 02, the 64 features are not manually defined by the programmer.

They are determined by learned parameters.

Key Idea

A hidden layer can be viewed as a learned transformation of the original representation.

What Does “Learned Representation” Mean?

Traditional ML often begins with:

$$x = [x_1, x_2, \dots, x_d].$$

The features may have explicit meanings such as:

- age;
- glucose;
- blood pressure;
- BMI;
- etc.

A neural network transforms them into

$$h = [h_1, h_2, \dots, h_k].$$

The hidden features generally do not have such direct human-readable meanings.

Forward Propagation

For a two-hidden-layer network:

$$h_1 = \text{ReLU}(W_1x + b_1),$$

$$h_2 = \text{ReLU}(W_2h_1 + b_2),$$

and

$$z = W_3h_2 + b_3.$$

The final vector z contains the output scores.

Thus:

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow z$$

This complete computation is called **forward propagation**.

Classification Output

For a C -class problem, the network produces

$$z = [z_1, \dots, z_C].$$

These are commonly called logits.

For probability interpretation:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}.$$

This is the softmax function.

The predicted class is commonly

$$\hat{y} = \arg \max_i z_i.$$

PyTorch Practice

When using `CrossEntropyLoss`, the network should normally return logits rather than applying softmax inside the model.

From Prediction to Learning

Forward propagation produces a prediction.
But how does the network learn?

Input $\rightarrow$ Prediction $\rightarrow$ Loss

The loss measures disagreement between the prediction and the target.
For classification:

$$\mathcal{L} = \text{CrossEntropy}(y, z).$$

The learning problem becomes:

$$\min_{\theta} \mathcal{L}(\theta)$$

Gradient Descent

Suppose θ represents all learnable parameters.

Gradient descent updates:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}.$$

where:

- η is the learning rate;
- $\nabla_{\theta} \mathcal{L}$ is the gradient.

The central idea is:

Change parameters $\rightarrow$ reduce loss

Backpropagation

Backpropagation computes gradients of the loss with respect to model parameters.

Conceptually:

Forward $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Update

The chain rule is fundamental.

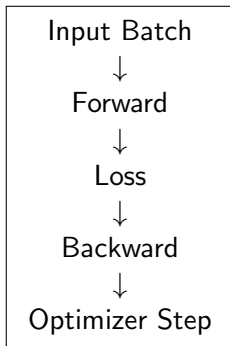
For example:

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial W_1}.$$

Students do not need to derive every gradient manually, but they must understand the role of the chain rule.

The PyTorch Training Cycle

A complete training step is:



In PyTorch:

```
optimizer.zero_grad()

outputs = model(X_batch)

loss = criterion(
```

Building the MLP in PyTorch

```
import torch
import torch.nn as nn

class MLP(nn.Module):

    def __init__(self, input_dim, num_classes):
        super().__init__()

        self.network = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, num_classes)
        )

    def forward(self, x):
        return self.network(x)
```

Question

What are the input and output dimensions of every layer?

Understanding Tensor Shapes

Suppose:

$$X \in \mathbb{R}^{32 \times d}.$$

The batch contains 32 observations.

After the first layer:

$$32 \times d \rightarrow 32 \times 64.$$

After the second layer:

$$32 \times 64 \rightarrow 32 \times 32.$$

Finally:

$$32 \times 32 \rightarrow 32 \times C.$$

Batch Dimension $\times$ Feature Dimension

Dataset and DataLoader

PyTorch commonly separates:

Dataset and DataLoader.

A Dataset stores or provides individual samples, while a DataLoader organizes samples into mini-batches.

Example:

```
from torch.utils.data import TensorDataset
from torch.utils.data import DataLoader

dataset = TensorDataset(
    X_tensor,
    y_tensor
)

loader = DataLoader(
    dataset,
    batch_size=32,
    shuffle=True
)
```

Why Mini-Batches?

Instead of processing all N observations simultaneously, training can use mini-batches.

$$X \rightarrow X_1, X_2, \dots, X_B.$$

For each batch:

$$\mathcal{L}_b = \mathcal{L}(X_b, y_b).$$

Advantages include:

- efficient memory usage;
- efficient computation;
- frequent parameter updates;
- compatibility with GPU computation.

Loss Function and Optimizer

For multi-class classification, a common loss function is:

$$\mathcal{L} = \text{CrossEntropyLoss}(z, y),$$

where z denotes the model logits and y is the target class.

In PyTorch:

```
criterion = nn.CrossEntropyLoss()

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001
)
```

The two components have different roles:

Loss Function → measures prediction error

Optimizer → updates model parameters

A Complete Training Loop

```
for epoch in range(num_epochs):  
    model.train()  
  
    for X_batch, y_batch in train_loader:  
        optimizer.zero_grad()  
  
        outputs = model(X_batch)  
  
        loss = criterion(  
            outputs,  
            y_batch  
        )  
  
        loss.backward()  
  
        optimizer.step()
```

Students must be able to explain every major operation.

Training vs. Evaluation Mode

PyTorch models have different operating modes.

During training:

```
model.train()
```

During validation or testing:

```
model.eval()
```

During evaluation, gradients are normally not required:

```
model.eval()

with torch.no_grad():
    outputs = model(X_test)
```

Important Distinction

`model.train()` and `model.eval()` control the model's operating mode. `torch.no_grad()` disables gradient computation for the enclosed operations.

Training Curves

Students should monitor both training and validation behavior.

Typical quantities include:

- training loss;
- validation loss;
- training accuracy;
- validation accuracy.

The curves help identify:

Underfitting

Good Fit

Overfitting

Important

A high training accuracy does not necessarily mean good generalization.

Underfitting and Overfitting

Underfitting

- training performance is poor;
- validation performance is also poor;
- model may have insufficient capacity;
- optimization may be inadequate.

Overfitting

- training performance becomes very high;
- validation performance stops improving;
- training and validation behavior diverges;
- model may memorize training patterns.

Generalization > Training Accuracy Alone

Evaluation

The final model should be evaluated on held-out test data.

For classification:

- Accuracy;
- Precision;
- Recall;
- F1-score;
- Confusion matrix.

Students should compare:

Training $\neq$ Validation $\neq$ Test

The test set should provide an unbiased estimate of final generalization under the experimental design.

Traditional ML vs. Neural Network

Suppose Assignment 02 produced:

Logistic Regression

and Assignment 03 produces:

MLP.

The important comparison is not only accuracy.

	Logistic Regression	MLP
Representation	Prepared	Learned
Model capacity	Lower	Higher
Nonlinearity	No	Yes
Parameters	Fewer	More
Interpretability	Higher	Lower
Training cost	Lower	Higher

A Scientific Model Comparison

A neural network should not automatically be considered better than traditional ML.

Students should ask:

- Is the dataset large enough?
- Is the learned representation useful?
- Does validation performance improve?
- Does test performance improve?
- How many parameters are introduced?
- What is the computational cost?
- Is the improvement practically meaningful?

Better Model $\neq$ Higher Accuracy Only

Inspecting the Learned Representation

Consider:

$$h = \text{ReLU}(W_1x + b_1).$$

Students can extract h from a hidden layer.

Then investigate:

$$x \in \mathbb{R}^d \longrightarrow h \in \mathbb{R}^k.$$

Questions:

- Has the dimensionality changed?
- Do observations from different classes separate?
- Which patterns appear in the learned space?
- What information may have been transformed?

Representation Is a Learned Transformation

Traditional ML:

$$x \rightarrow \text{ML Algorithm} \rightarrow \hat{y}$$

Deep learning:

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow \cdots \rightarrow \hat{y}$$

Each hidden layer applies a learned transformation.

Therefore:

Deep Network = Composition of Learned Transformations

Saving a PyTorch Model

After training, the learned parameters can be saved.

```
torch.save(  
    model.state_dict(),  
    "mlp_model.pth"  
)
```

Later, reconstruct the same architecture and load the parameters:

```
model = MLP(  
    input_dim=input_dim,  
    num_classes=num_classes  
)  
  
model.load_state_dict(  
    torch.load("mlp_model.pth")  
)  
  
model.eval()
```

Key Idea

`state_dict()` stores the learned parameter state. The model architecture

Assignment 03: Required Experiments

Students must conduct at least three controlled experiments.

1 Baseline MLP

$$d \rightarrow 64 \rightarrow C$$

2 Deeper MLP

$$d \rightarrow 64 \rightarrow 32 \rightarrow C$$

3 Optimization Experiment

Compare at least two learning rates or optimizers.

For each experiment report:

- architecture;
- number of parameters;
- training configuration;
- training/validation curves;
- test metrics;
- interpretation.

Assignment 03: Required Deliverables

Students must submit:

- 1 **Jupyter Notebook**
- 2 **PyTorch Source Code**
- 3 **Saved Model**
- 4 **Experimental Results**
- 5 **Technical Report**
- 6 **Short Presentation**

The notebook should run from beginning to end.

Academic Requirement

Students must be able to explain the architecture, tensor shapes, loss, optimizer, training loop, evaluation, and experimental conclusions.

Academic Use of AI Assistants

AI assistants may be used as **learning and development tools**.

Students remain responsible for their submitted work.

They must understand:

- tensor shapes;
- network architecture;
- forward propagation;
- loss function;
- optimizer;
- backpropagation;
- training loop;
- evaluation;
- important source code.

Principle

AI may assist development, but understanding must belong to the student.

Deep learning learns representations together with the prediction function.

Input $\rightarrow$ Learned Representation $\rightarrow$ Prediction

But a neural network is still a mathematical model:

Layers + Parameters + Loss + Optimization

Looking Forward: From Vectors to Images

So far:

$$x \in \mathbb{R}^d$$

is a feature vector.

But an image is naturally represented as a tensor:

$$X \in \mathbb{R}^{C \times H \times W}.$$

For example:

$$3 \times 224 \times 224.$$

A fully connected MLP can process the flattened vector, but this may ignore important spatial structure.

The next question is:

How can a neural network learn spatial representations?

From MLP to CNN

The next assignment introduces convolutional neural networks.

Vector $\rightarrow$ MLP

becomes

Image Tensor $\rightarrow$ CNN

The representation now preserves spatial organization.

Deep Learning $\rightarrow$ Spatial Representation Learning

Next

Assignment 04: Convolutional Neural Networks and image representation learning.